

From script to web app and API

One saved spam classifier: a page for people with Streamlit or Gradio, an endpoint for programs with FastAPI.

THE QUESTION

How does a teammate try your model in a browser, and how does another program call it?

THE INTUITION

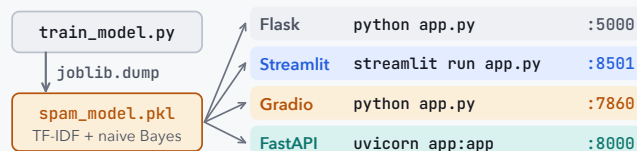
Save the model to a file and load it once. Streamlit and Gradio draw a page around Python code; FastAPI turns a function into a JSON URL.

KEY TAKEAWAY

Load the model once, not per click. Gradio for demos, Streamlit for dashboards, FastAPI when code calls the model.

notebook → joblib.dump → FastAPI: programs → Gradio / Streamlit: people → Docker → Hugging Face Spaces: anyone

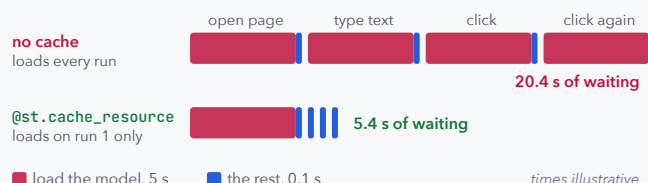
1 Save the pipeline once; every app loads it



each app.py calls joblib.load; FastAPI answers with JSON, the others draw a page

joblib.dump keeps the TF-IDF step and the classifier in one file. PyTorch saves a state_dict with torch.save; ONNX runs in other languages too.

2 Every interaction reruns the whole script



```
@st.cache_resource
def load_model():
    return joblib.load("spam_model.pkl")

model = load_model()
text = st.text_area("Message")
if st.button("Classify"):
    p = model.predict_proba([text])[0]
    st.write(f"Spam: {p[1]:.1%}")
```

@st.cache_resource runs load_model once; later runs get the same object. Save app.py and the page offers to rerun.

3 A widget call returns its current value

	run 1 open page	run 2 move slider	run 3 click Predict	run 4 pick Comedy
st.number_input("Budget")	50.0	50.0	50.0	50.0
st.slider("Runtime")	120	150	150	150
st.selectbox("Genre")	"Action"	"Action"	"Action"	"Comedy"
st.button("Predict")	False	False	True	False
result message	none	none	Hit! / Flop	gone

st.button is True only on the run it's own click caused

st.slider("Runtime (min)", 60, 240, 120) returns 120 until the user moves it. A result shown under if st.button(...) vanishes at the next interaction. Layout: st.columns, st.sidebar, st.tabs; output: st.success, st.error, st.metric, st.dataframe, st.line_chart.

4 Session state keeps values between reruns

	Save idea 1 typed "OCR app"	Save idea 2 typed "RAG bot"	Save idea 3 typed "quiz maker"
ideas = [] resets every run	OCR app	RAG bot	quiz maker
st.session_state survives reruns	OCR app	OCR app RAG bot	OCR app RAG bot quiz maker

```
if "ideas" not in st.session_state:
    st.session_state.ideas = []
new = st.text_input("Add an idea")
if st.button("Save idea"):
    st.session_state.ideas.append(new)
```

A plain variable is rebuilt every run; st.session_state is not. One per browser tab.

5 cache_data keeps one result per argument

	run 1	run 2	run 3	run 4	run 5	run 6
slider m	1	2	1	3	2	1
build_schedule	runs	runs	cached	runs	cached	cached
total hours	15	30	15	45	30	15

3 of 6 calls ran the function (slider moves illustrative)

```
@st.cache_data
def build_schedule(m): ... # DataFrame
m = st.sidebar.slider("Scale", 1, 4, 1)
df = build_schedule(m)
```

Each caller gets a copy; cache_resource shares one object (models).

6 Gradio wraps one function in a page



```
def classify(text):
    p = model.predict_proba([text])[0]
    return {"Not Spam": float(p[0]),
            "Spam": float(p[1])}

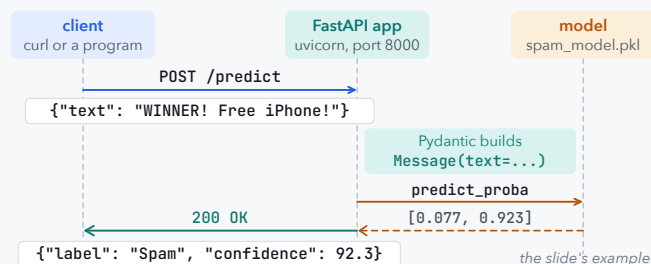
demo = gr.Interface(fn=classify,
                    inputs=gr.Textbox(label="Message"),
                    outputs=gr.Label(label="Prediction"))
demo.launch() # port 7860
```

Also gr.Image, gr.Audio, gr.Sketchpad; code calls it via gradio_client.

An endpoint for programs, a link for people

FastAPI turns a typed function into a URL; Pydantic checks the JSON first. Spaces and Docker put the app online.

7 FastAPI turns a typed function into a URL



```
app = FastAPI(title="Spam Classifier API")
model = joblib.load("spam_model.pkl")
class Message(BaseModel):
    text: str
```

```
@app.post("/predict")
def predict(msg: Message):
    p = model.predict_proba([msg.text])[0]
    pred = model.predict([msg.text])[0]
    return {
        "label": "Spam" if pred == 1
            else "Not Spam",
        "confidence": round(
            float(max(p)) * 100, 1)}

```

Start it with `uvicorn app:app --reload --port 8000` (the object `app` in `app.py`). `/docs` serves a Swagger page built from the Pydantic models. The client side of HTTP is on the HTTP and APIs sheet.

8 Pydantic answers bad input with a 422

POST {"budget": -5} → 422 Unprocessable Entity

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "budget"],
      "msg": "Input should be greater than 0",
      "input": -5,
      "ctx": {"gt": 0.0},
      "type": "missing",
      "loc": ["body", "runtime", ...],
      ... 3 more "missing"
    }
  ]
}
```

type: a code for programs
loc: body, then the field
msg: text for people
input: what was sent

5 errors in one reply: budget breaks `gt=0`; 4 fields are missing

```
class MovieFeatures(BaseModel):
    budget: float = Field(..., gt=0)
    runtime: float = Field(..., gt=0, le=1)
    # + three more fields, each 0 to 1
```

... marks a required field; `gt`, `ge`, `lt`, `le` set bounds; `predict` never runs. Pydantic v1 wrote "ensure this value is greater than 0". More on the data validation sheet.

9 Call it with curl; test it without a server

```
curl -X POST localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "WINNER! Free iPhone!"}'
```

```
client = TestClient(app) # in-process
def test_predict_invalid():
    r = client.post("/predict",
                    json={"budget": -1})
    assert r.status_code == 422
```

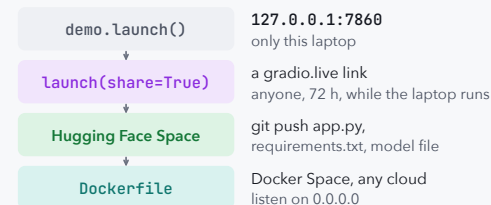
A `List[MovieFeatures]` body gives a batch endpoint.

10 Pick the tool by who uses the model

	Streamlit	Gradio	FastAPI
you write	a script	a function	typed functions
each event runs	the whole script	the function	one endpoint
used by	people	people	programs
the page	from <code>st.*</code> calls	from <code>gr.*</code> parts	none; <code>/docs</code>
API for code	no	yes, automatic	yes, JSON
default port	8501	7860	8000
best for	dashboards	model demos	production APIs

Flask, a fourth option, has you write the routes and HTML.

11 A Space for demos, a container for APIs



```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
COPY model.pkl app.py .
CMD ["uvicorn", "app:app", "--host",
    "0.0.0.0", "--port", "8000"]
```

Commit the model file (no `../0-model/` on a Space) and list every package in `requirements.txt`, or the build fails with `ModuleNotFoundError`. A Docker Space serves port 7860 unless its README sets `app_port`. Ports: the reproducible runs sheet.

12 Pause and think

Each click takes 5 seconds? The model loads with no cache; use `@st.cache_resource`.

400 or 422? 422: the body failed validation. 404: unknown path. 405: wrong method.

Online or batch? A waiting user needs about 100 ms: an API. Overnight scoring: a cron script.

load once · `cache_resource` for models · `session_state` for memory · Pydantic per endpoint · ship `requirements.txt`

Source: lectures "From script to web app" and "APIs and model demos", their web-app demos, the Streamlit tutorial apps and the APIs follow-along, nipunbatra.github.io/stt-ai-teaching. The 422 body is FastAPI's own reply (TestClient, FastAPI 0.139, Pydantic 2.13); 92.3% is the slide's example; rerun times and slider moves are illustrative. Next: the profiling and quantization sheet.